

a) Edgar Fabricio Rios Cuenca

Paso 2:

Ejercicio 1:

Paso 2:

Los estudiantes cuestionan y resuelven de forma Participativa las leyes y propiedades adicionales mediante ejemplos prácticos conectando cada propiedad con aplicaciones reales en diseño digital.

1) Consenso Dual.

$$F = (A+B) (\bar{A}+C) (B+C) (D+C)$$
$$\text{Consenso Dual} = (B+C)$$
$$\text{Eliminar} = (B+C)$$
$$F = (A+B) (\bar{A}+B) (D+C) //$$

Ejercicio 2:

2.- Expansión de Shanon.

Función $\rightarrow F(A,B,C) = AB + A\bar{C} + \bar{B}C$

Evaluar $A = 1 \rightarrow F(1,B,C) = (1)B + (1)\bar{C} + \bar{B}C$
 $F(1,B,C) = B + \bar{C} + \bar{B}C$
 $F(1,B,C) = B + C + \bar{C}$
 $F(1,B,C) = 1$

Evaluar $A = 0 \rightarrow F(0,B,C) = (0)B + (0)\bar{C} + \bar{B}C$
 $F(0,B,C) = \bar{B}C$

Resultado Final $\rightarrow F = A(1) + \bar{A}(\bar{B}C)$
 $F = A + \bar{A}\bar{B}C$
 $F = A + \bar{B}C //$

Ejercicio 3:

3. Resolución Booleana

- ▶ Clausula 1: $x + y$
(o es un problema de software, o es de hardware).
- ▶ Clausula 2: $\bar{x} + y$
(Si no es un problema de Software, entonces se reinicia).
- ▶ Clausula 3: $y + w$
(si no es un problema de hardware, entonces se activa la alerta).
- ▶ Clausula 4: $\bar{z}$
(El sistema no se reinicia).
- ▶ Resolvente 1: Entre 2 y 4:
$$\begin{array}{r} (\bar{x} + \bar{z}) \quad (\bar{z}) \\ \bar{x} \end{array}$$
- ▶ Resolvente 2: Entre Resolvente 1 y 1:
$$\begin{array}{r} (\bar{x} + \bar{z}) \quad (\bar{x}) \\ \bar{z} \end{array}$$
- ▶ Resolvente 3: Entre Resolvente 2 y 3:
$$\begin{array}{r} (\bar{y} + w) \quad (y) \\ w \end{array}$$

$w \rightarrow$ Se activa la alerta

Paso 3:

Ejercicio 1:

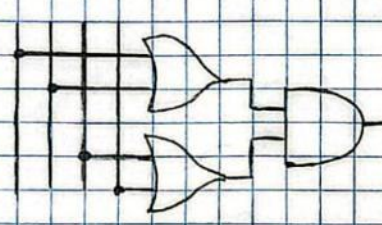
Paso 3: Práctica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR (puertas universales). Los estudiantes diseñan circuitos equivalentes usando un tipo de compuertas.

• Ejercicio 1:

Función Original:

$$F = (A+B)(C+D)$$

A B C D



Función final NAND:

$$F = (A+B)(C+D) \rightarrow \text{Ley de doble Negación.}$$

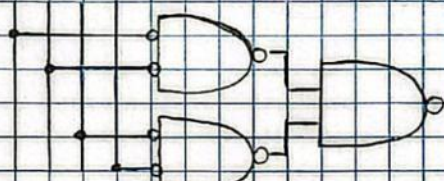
$$\overline{\overline{(A+B)(C+D)}} \rightarrow \text{Ley de Morgan}$$

$$\overline{(A+B) \cdot (C+D)} \rightarrow \text{Ley de Morgan}$$

$$\overline{A+B} \cdot \overline{C+D}$$

$$(\overline{A+B})(\overline{C+D}) //$$

A B C D



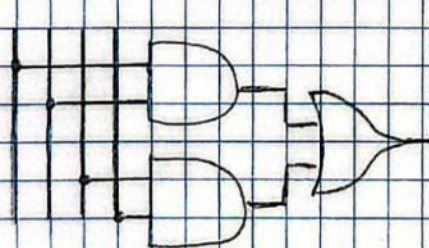
Ejercicio 2:

• Ejercicio 2:

Función Original:

$$F = AB + CD$$

A B C D



Función final NOR:

$$F = AB + CD \rightarrow \text{Ley de doble Negación}$$

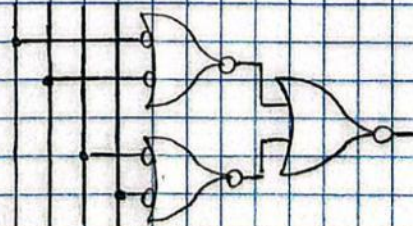
$$\overline{\overline{AB + CD}} \rightarrow \text{Ley de Morgan}$$

$$\overline{(A \cdot B) + (C \cdot D)} \rightarrow \text{Ley de Morgan}$$

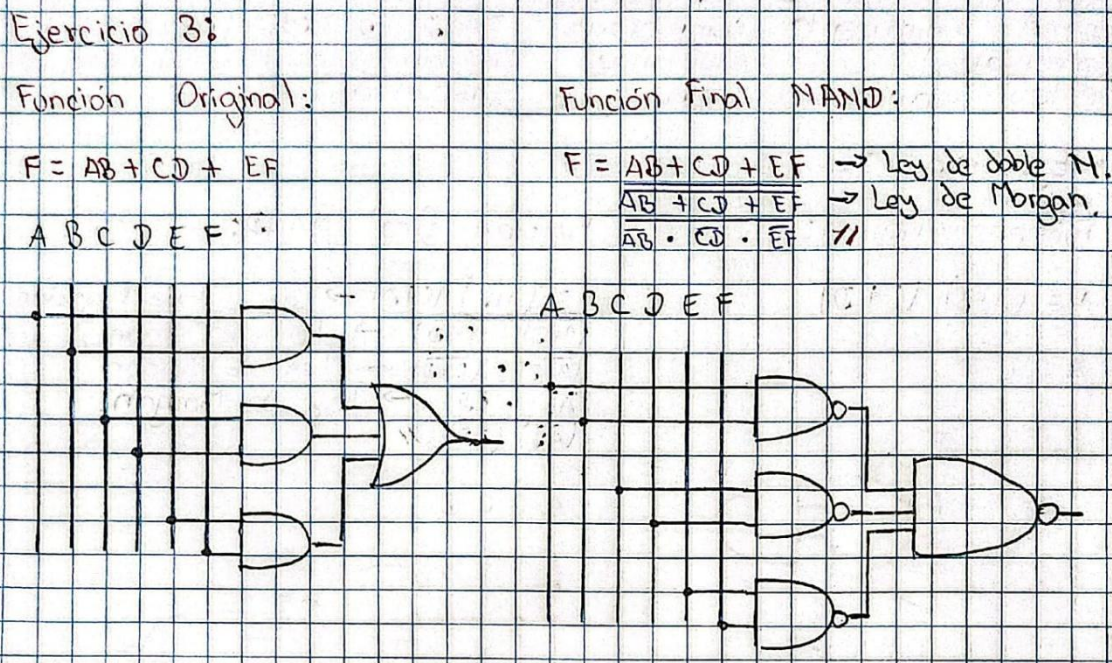
$$\overline{(A+B) \cdot (C+D)}$$

$$(\overline{A+B})(\overline{C+D}) //$$

A B C D



Ejercicio 3:



Paso 4:

Paso 4:

Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales, analizando como la simplificación booleana reduce costos, consumo energético y huella ambiental en la fabricación de circuitos.

Informe reflexivo sobre la optimización lógica en sistemas computacionales.

En el diseño de sistemas computacionales modernos, la eficiencia ya no se mide únicamente en términos de velocidad de procesamiento, sino también en función de la optimización de los recursos materiales y energéticos. El presente informe expone nuestra reflexión colectiva sobre el impacto de la optimización lógica y la simplificación booleana en la ingeniería de circuitos. A lo largo de este análisis, demostramos cómo las decisiones de diseño matemático en el software abstracto se traducen directamente en beneficios económicos y ecológicos tangibles en el mundo físico.

Análisis de la simplificación booleana.

La simplificación booleana (mediante métodos como los Mapas de Karnaugh) permite reducir al mínimo el número de compuertas lógicas e interconexiones necesarias para ejecutar la función. Este impacta positivamente en tres ejes críticos:

1.- Reducción de Costos de fabricación:

- Al reducir las compuertas necesarias, disminuye el área de silicio requerida para cada circuito integrado. Esto permite crear más microchips por cada oblea de producción.
- Los circuitos más pequeños y menos complejos tienen una menor tasa de defectos físicos durante el proceso de fotolitografía, lo que reduce el desperdicio de materiales costosos y maximizar el rendimiento (yield) de la fábrica.

2.- Consumo Energético:

- Cada compuerta lógica consume energía eléctrica y disipa calor cada vez que cambia de estado. Eliminar componentes redundantes reduce la capacitación global y las corrientes de fuga del circuito.
- Un menor consumo de energía disminuye la temperatura operativa del dispositivo, reduciendo o eliminando la necesidad de sistemas de enfriamiento activos (como ventiladores), lo que genera un ahorro energético secundario.

3. Huella Ambiental:

- Circuitos más pequeños significan una menor demanda de materias primas críticas, como silicio de alta pureza, cobre y tierras raras, mitigando el impacto de la minería intensiva.
- La reducción del tamaño de los componentes disminuye el uso de químicos peligrosos en las etapas de grabado (etching) y lavado en las plantas de semiconductores.
- A largo plazo, millones de dispositivos optimizados operando globalmente reducen de manera significativa la demanda eléctrica y las emisiones indirectas de CO_2 .

Conclusión Reflexiva.

Como equipo, concluimos que la optimización de sistemas lógicos no es solo un ejercicio de eficiencia técnica, sino una práctica de ingeniería responsable. La simplificación de una ecuación booleana en la fase de diseño de software tiene un impacto en el ecosistema, demostrando que la optimización matemática es una de las herramientas más limpias para reducir costos industriales y mitigar la huella ecológica de la era digital.

Paso 5:

5.- Caso aplicado a la computación: Sumador binario de 4 bit

1: Tabla de verdad.

Entrada: A, B y Acarreo de entrada (C_{in})

Salida: Suma (S) y Acarreo de Salida (C_{out})

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2.- Simplificación de la Suma.

$$SOP: S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC$$

$$\bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC \rightarrow \text{Ley Distributiva}$$

$$\bar{A}(\bar{B}C_{in} + B\bar{C}_{in}) + A(\bar{B}\bar{C}_{in} + BC_{in}) \rightarrow \text{Sustitución}$$

$$\bar{A}(B \oplus C_{in}) + A(B \oplus C_{in}) \rightarrow \text{Definimos variable auxiliar } B \oplus C_{in} = X$$

$$\bar{A}X + AX \rightarrow \text{Sustitución}$$

$$A \oplus X \rightarrow \text{Reemplazo del auxiliar.}$$

$$A \oplus B \oplus C_{in}$$

(Para sustituciones definimos).

Definición de XOR:

$$X \oplus Y = \bar{X}Y + X\bar{Y}$$

Definición de XNOR

$$X \odot Y = \overline{X \oplus Y} = \bar{X}\bar{Y} + XY$$

3. Simplificación de salida de acarreo (Cout)

$$SOP = Cout = \bar{A}BCin + A\bar{B}Cin + AB\bar{C}in + ABCin$$

$$\bar{A}BCin + A\bar{B}Cin + AB\bar{C}in + ABCin \rightarrow \text{Ley distributiva.}$$

$$Cin (\bar{A}B + A\bar{B}) + AB (\bar{C}in + Cin) \rightarrow \text{Ley de Complemento}$$

$$Cin (\bar{A}B + A\bar{B}) + AB (1) \rightarrow \text{Ley de identidad.}$$

$$Cin (\bar{A}B + A\bar{B}) + AB \rightarrow \text{Sustitución.}$$

$$Cin (A \oplus B) + AB$$

4. Diseño de las compuertas lógicas.

